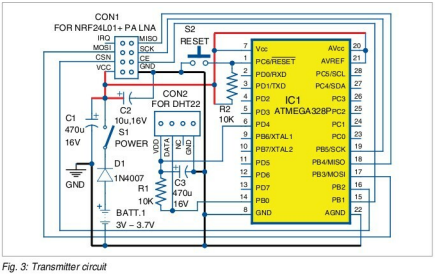

Fig. 3: Transmitter circuit

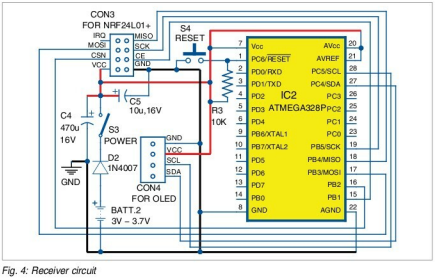

Fig. 4: Receiver circuit

is tricky. At the same time, getting around 1µA in deep-sleep mode is easy, with the help of the chip's watchdog timer (WDT) and brown-out detection fuse.

First, turn off the brown-out detection fuse so that low-voltage operation becomes possible without resetting. As the voltage goes down, so does the current (refer the graph in Fig. 2).

Interestingly, as the frequency goes down, so does the power consumption. That means if the chip is designed to work on a lower resonating frequency, the power consumption will further reduce. To reduce the frequency of operation, we divide the 8MHz frequency internally by commands. At 3.3V operation, power consumption goes down as follows:

```
clock_div_1-3.1 mA clock_div_2 - 1.8 mA
clock_div_4-1.1 mA clock_div_8 - 750 µA
clock_div_16-550 µA cock_div_32 - 393 µA
clock_div_64-351 µA clock_div_128 - 296 µA
clock_div_256 - 288 µA
```

To reduce the frequency of operation, here's a very simple command to use inside setup( ):

```
// slow clock to divide by 256
clock_prescale_set (clock_div_256);
```

As per Fig. 2, at 4MHz, the chip will continue to work off 1.8V supply.

If the WDT is turned off, the chip will get sleep current to the tune of 1µA, but it will not wake up on its own—unless you give it a shock through interrupter (0) from pin 4 of ATmega328P, momentarily making it ground.

However, in this project we want it to send signals periodically (once every 42 minutes). Therefore we don't set the WDT 'off.' The sleep current will be more than 30µA but the operation will be periodic on its own. The WDT clock is not very precise as compared to other methods of time keeping. However, at low power, the WDT clock does not differ much from real-time clocks. The entire operation is accomplished with the help of a library file called lowpower.h, which can be downloaded from the following link: *http://www.rocketscream.com/blog/2011/07/04/lightweight-low-power-arduino-library/*

Transmitter unit

Fig. 3 shows the transmitter circuit. It can be powered by a 3.7V Li-ion cell or two 1.5V pencil cells. Theoretically, the devices used here are capable of running at up to 1.8 volts, but at less than 2 volts the nRF24L01+PA&LNA radio stops working and at less than 2.8 volts the DHT22 sensor will not function. Small capacitors are added to the supply bus of the DHT22 and the nRF24L01 for stability. Vcc supply for the DHT22 is taken from port PD4 (pin 6) of ATmega328P, whereas the nRF24L01+ is directly connected with 3.7V supply. To reduce power consumption by the radio in idle state, we use powerDown( ) command of the radio in the code.

Before burning the code (nrf24l01_tx6.ino) into a fresh ATmega328P chip, burn the bootloader code for the inbuilt internal 8MHz clock. For details, see under AVR programmer sub-head.

Receiver unit

Fig. 4 shows the receiver circuit. The nRF24L01+ radio is designed